UniPay

Unified Payment, Identity & Reconciliation Platform

One identity. Every rail. One ledger. For everyone.

Version 4.0 — Updated Technical Documentation
Phase 1 Scope: Kenya · KES · LOOP Rail · Small Businesses & Individuals
React (Web + Mobile-responsive) + Clerk + LOOP + AI Layer
13 August 2026

Table of Contents

1. Executive Summary

UniPay (formerly PesaLink) gives any Kenyan individual or small business — a market vendor, a freelancer, a boda rider, a family member sending a friend money, an online seller — one shareable identity (an alias, a QR code, or a link) through which anyone can pay them. In this phase, every payment moves in one currency, Kenyan Shillings (KES), over one live rail (LOOP), and lands in one normalized ledger with clear fees, clear settlement status, and automatic reconciliation.

The single biggest shift in this version: UniPay is no longer a merchant-only tool. It is a general-population payments platform. A merchant and an individual customer use the exact same account model, the same alias system, and the same app — the only difference is whether a profile is flagged as a business. This makes UniPay usable by literally anyone with a phone number, not just people who sell things.

Unification is still the core idea everything else serves. The problem was never a shortage of ways to pay in Kenya — it's that those ways are scattered across disconnected identifiers, dashboards, and now, disconnected apps for "consumer" and "merchant" use cases that don't need to be separate at all. UniPay collapses that: one system, one identity, one ledger, one app that renders naturally on a phone or a laptop, whether you are a shopper, a shop owner, or a sibling sending rent money home.

This phase deliberately narrows scope on **currency and geography** (KES only, Kenya only) so that the team can go deep on **breadth of user base** (merchants and individuals), **platform intelligence** (a genuine AI layer, not a chatbot bolted on at the end), **administrative control** (a real Admin module), and **user choice over money** (configurable money-direction routing). Multi-currency and cross-border rails (PayPal, international cards) remain fully compatible with the architecture and are the clearly-scoped next phase — see Section 25.

2. System Description

UniPay is a web-first, mobile-responsive payments platform that gives any person or small business in Kenya a single, verifiable identity to send and receive money, see exactly where that money stands at every moment (paid, settled, or withdrawn), and decide where it goes next.

At its core, UniPay is three systems working as one:

- **An identity layer** — every user, individual or business, registers once and receives one alias, QR code, and payment link. Identity is lightly verified, so the person on the other end of a payment is provably who they claim to be.
- **A payment orchestration layer** — every transaction, regardless of how it was initiated, is normalized into one internal shape and tracked through independent payment, settlement, and disbursement states, so nobody is ever confused about what money is real, settled, and spendable.
- **An intelligence layer** — an AI service sits across reconciliation, fraud detection, support, and reporting, doing the pattern-matching and language-understanding work a human would otherwise have to do manually (see Section 15).

Above these three layers sits a fourth: an Admin module that gives the UniPay operations team visibility and control over the whole platform — users, transactions, exceptions, and configuration — without ever touching a database directly (see Section 16).

The result is a platform that answers, for anyone, the same three questions a payments app should always be able to answer instantly: Did I get paid? Is that money actually mine to spend yet? And if it is, where do I want it to go?

3. Problem Statement

Kenyans — not just registered merchants, but ordinary people, freelancers, and small sellers — operate across a fractured identity and reconciliation landscape:

- A single person or small business may have a till number, a paybill, a personal phone number, and a bank account — with no single front door for the other party to use.
- Customers and everyday senders frequently don't know which identifier to use for a given person or business, causing failed, delayed, or abandoned payments.
- Fees are opaque until after a payment has already been committed to.
- A successful payment and a settled payment are not the same event — recipients routinely can't tell whether money has actually landed somewhere they can spend it from.
- Reconciling payments against orders, invoices, or even simple personal expectations is done manually — in notebooks, WhatsApp threads, or disconnected mobile-money statements — which is slow and leaks money through missed or mismatched payments.
- Existing tools split the world into "consumer money apps" and "merchant payment gateways," forcing a small seller (or a person who occasionally sells things informally) to juggle two different mental models and, often, two different apps.

4. The Challenge

Build a working demonstration of a solution to the above that:

- Is technically real (not just a mockup) on the live payment rail
- Works identically well for an individual sending money to a friend and a small business collecting from customers
- Clearly, demonstrably solves the reconciliation and visibility problem — not just "accepts payments"
- Gives users real control over where their money ends up, not just a static wallet balance
- Meaningfully integrates AI in a way that changes what the product can do, not as a gimmick layered on top
- Is differentiated against both traditional mobile-money tooling and typical fintech orchestration platforms

5. The Solution

UniPay lets anyone — an individual or a business — register once and receive a single UniPay Alias (e.g. @amina or @aminagroceries), a QR code, and a payment link. Anyone paying them completes checkout through the same branded flow, in KES, over the LOOP rail. Behind that single flow:

- A provider adapter layer normalizes every payment into one internal transaction shape, ready for additional rails and currencies without rearchitecture
- A fee transparency layer shows the payer exactly what they're paying and the recipient exactly what they'll net, before money moves
- A payment-vs-settlement state machine tracks each transaction through two independent lifecycles, so nobody is ever confused about what's spendable
- A reconciliation engine, assisted by AI matching, auto-matches transactions to orders or expected payments using reference, amount, and time-window heuristics, escalating only genuine exceptions to a human
- A money-direction setting lets every user choose where settled funds go next, and change that choice at any time
- A unified dashboard gives one real-time view across every transaction, whether the user is an individual or a business
- An Admin module gives the platform operator oversight of users, transactions, exceptions, and configuration

What makes it unique ("the mind-blowing part")

- **One identity, one app, everyone.** There is no separate "merchant app" and "consumer app." A single account model serves a market vendor and her customer equally — the platform adapts to the account, not the other way around.
- **Settlement honesty.** UniPay is explicit that "paid" ≠ "settled" ≠ "in your chosen destination," which is the single most common source of cash-flow confusion in mobile-money commerce, and almost no consumer-facing tool surfaces this distinction clearly.
- **Confidence-scored, AI-assisted reconciliation.** Rather than a flat matched/unmatched binary, every match carries a confidence score and a human-readable reason generated with AI assistance, so users trust the system instead of re-checking it.
- **Money direction as a first-class feature.** Where settled money goes is a user decision, not a platform default — and it's changeable at any time from Settings, not buried in a one-time onboarding choice.
- **Adapter-first, currency-ready architecture.** Adding a new rail or a new currency is a new adapter and a config row, not a rewrite — proven by the same seeded mock-rail pattern used in earlier versions of this architecture.

6. Unified Objective

Give every person and small business in Kenya one trusted identity to send and receive money — and one truthful ledger that tells them exactly what they earned, what they'll receive, when, and where it's going — on a platform built for everyone, not just merchants, and ready to keep absorbing new rails, currencies, and intelligence without ever breaking that unity.

Every feature decision in this document is filtered through that sentence. If a feature doesn't strengthen unification, transparency, trust, or user control, it is out of scope for this phase.

7. Target Users

UniPay is deliberately scoped for the general population, not a merchant-only segment. Every persona below uses the same underlying account type — a UniPay Profile — flagged as either Individual or Business.

Persona	Profile	Core Need
Amina — Market Vendor (Business)	Sells groceries in Nairobi, currently uses till number + notebook	One identifier, instant confirmation, daily sales visibility, exportable record
Brian — Freelancer (Business/Individual)	Does design/dev work for local clients	Invoice-to-payment matching, knowing when a client payment actually clears
Ken — General Consumer (Individual)	Everyday person paying for goods, splitting bills, or sending money to family	A simple, trusted way to pay or receive money without needing a "business" account or learning till numbers
Njeri — Online Seller (Business)	Sells via Instagram/WhatsApp, shares a payment link per order	Needs a link/QR that "just works" regardless of who's paying
UniPay Admin — Operations Staff	Internal platform operator	Full visibility into users, transactions, exceptions, and rail/config health, with the tools to intervene safely

Note: the diaspora/international-customer persona from earlier versions of this document is retained conceptually but moves to the Phase 2 roadmap (Section 25) alongside multi-currency support, since this phase is intentionally KES-only.

8. Current Phase Scope

Phase 1 boundary: one country (Kenya), one currency (KES), one live rail (LOOP), open to both individuals and small businesses. Everything below is scoped to be real and demoable within that boundary, while the architecture underneath is built to outgrow it without rearchitecture (Section 9b).

Must-Build (Core Platform)

- Universal sign-up/login via Clerk — one flow for individuals and businesses, differentiated by an account-type flag, not a separate app
- Onboarding: profile details + ID upload + ID number capture (lightweight identity verification, Section 15b-equivalent)
- One alias + QR generation per user, gated on identity submission (verification_status)
- Real LOOP sandbox request-to-pay integration (init → webhook/poll → normalized transaction), KES only
- Checkout page that resolves an alias and completes payment in KES
- Normalized transaction ledger
- Payment-status vs settlement-status displayed distinctly per transaction
- Rule-based reconciliation against expected orders/payments (Section 14)
- AI-generated match explanations — one LLM call per match, turning the confidence score into a plain-language reason (Section 15, Priority 0 #1)
- AI-powered natural-language dashboard queries — a search box that answers questions like "how much did I make last week" against the transaction ledger (Section 15, Priority 0 #2)
- Money-direction setting: users choose a default destination for settled funds and can change it any time in Settings
- Disbursement: users can request payout of their available balance, tracked through a real payouts table, calling the LOOP payout/B2C endpoint where the sandbox allows
- Unified dashboard: totals, available-to-withdraw balance, exceptions list — same dashboard shape for an individual and a business, with business-only widgets (e.g. per-product breakdown) shown conditionally
- Admin module: user management, identity review queue, transaction/exception monitoring, rail configuration (Section 16)
- One CSV export

Should-Build (if time remains)

- Exception review UI (approve/reject a low-confidence match)
- Notification (toast/SMS) on payment received
- Admin-side manual approve/reject on submitted ID verification (otherwise auto-approve for demo purposes)
- AI anomaly/fraud flagging on the exception queue (Section 15, Priority 1)
- AI-assisted smart rail routing recommendation (Section 15, Priority 1)
- AI ID-document pre-check — legibility and field-consistency triage before the review queue (Section 15, Priority 1)
- AI-generated weekly summary / receipt text (Section 15, Priority 1)

Simulated / Seeded (never live-coded, but shown to prove the architecture scales)

- A second currency and a second rail, seeded through the exact same PaymentProviderAdapter and payment_rails config pattern, proving the multi-currency/multi-rail claim without needing a live integration in this phase
- Historical settlement and payout records for realism in the dashboard

- ID verification "checked against registry" result — submission and status tracking are real; the actual registry check is seeded/simulated, since integrating a licensed ID-verification provider is out of scope for this phase

Explicitly out of scope for this phase: multi-currency/cross-border rails, production-grade KYC/registry integration, production card acquiring, automated lending, tax filing, holding customer funds beyond transient settlement, and real payout to arbitrary bank accounts (payout is scoped to LOOP-supported destinations only).

9. Solution Architecture

```
User Onboarding (Clerk sign-up)
  |  account_type: individual | business
  v
Identity Verification Step ---> ID Upload + ID Number ---> AI Pre-Check ---> verification_status
  |
  v
UniPay Profile + Alias/QR (gated on submission)

Payer Checkout
  |
  v
Alias Resolver ---> Recipient Profile
  |
  v
Fee Estimate Service
  |
  v
Payment Orchestrator
  |            |
  v            v
LOOP Adapter   Seeded/Mock Adapter (future-rail proof)
  |            |
  v            v
LOOP API     Fixture
  |            |
  +--> Webhook / Status Poller <--+
         |
         v
  Normalized Transaction Ledger
    |        |          |            |
    v        v          v            v
 AI-Assisted  Unified   Money-Direction  Admin Module
 Reconciliation Dashboard  Router          (oversight)
    |                       |
    v                       v
 Exception Queue      Disbursement Orchestrator
                             |
                             v
                    LOOP Payout Adapter ---> Payout Record
```

Recommended Stack

Layer	Technology	Notes
Frontend	React (Vite), Tailwind CSS	Single responsive codebase; renders as a full dashboard on desktop and a mobile-optimized layout on phones via Tailwind breakpoints — no separate mobile app (Section 20)
Auth	Clerk	One account model for individuals and businesses; account_type differentiates behavior, not the login flow

Backend	Node.js (Express/Fastify)	Exposes REST API consumed by the React frontend
Database	PostgreSQL (Supabase)	Also simplifies auth-adjacent metadata sync from Clerk via webhook
Payment Rails	LOOP API (real), seeded second rail (proof of scalability)	Both behind the adapter interface
AI Layer	LLM API (e.g. Claude) via a dedicated AI service	Reconciliation assistance, natural-language queries, document pre-check, support chat, anomaly flags — see Section 15
Queues	In-memory/cron-poll for this phase; Redis/BullMQ documented as the production path	Full queue infra is a poor use of build time at this stage
Hosting	Vercel (frontend), Render/Railway (backend)	Fast to deploy, free-tier friendly

9b. Scalability: Rails, Currencies & Segments

Because unification is the core idea, UniPay has to make adding a new rail, a new currency, or a new user segment cheap — otherwise "unified" only holds true for the one rail and one currency built in this phase. Three structural choices make this true:

- **Every rail is an implementation of PaymentProviderAdapter.** Checkout, the ledger, reconciliation, and the dashboard never contain rail-specific logic. Adding a bank A2A rail, a second mobile-money provider, or a card processor means writing one new adapter class and nothing else changes.
- **Rails and currencies are configuration, not a code deploy.** A payment_rails table (id, name, adapter_key, is_enabled, supported_currencies, supported_countries, capabilities_json) drives which rails and currencies appear at checkout. Enabling KES-only today and USD tomorrow is a database update, not a redeploy.
- **Account type is a flag, not a fork.** Individual and Business are both rows in the same profiles table with a shared schema and an account_type column. Business-only capabilities (e.g. staff sub-accounts, catalog-linked reconciliation) are additive fields and conditional UI, not a separate codebase — which is exactly what let this phase drop the "merchants only" restriction without a rewrite.

This is what makes the "unified" claim durable rather than a one-time demo trick: the platform's job, structurally, is to keep absorbing new rails, new currencies, and new kinds of users behind one identity and one ledger.

10. Provider Adapter Architecture

Every rail — real or mocked — implements the same contract, so the checkout, ledger, and reconciliation code never contains rail-specific logic:

```
interface PaymentProviderAdapter {
  name(): string;
  capabilities(): ProviderCapabilities;
  createPayment(request: PaymentRequest): Promise<ProviderPaymentResult>;
  getStatus(providerReference: string): Promise<ProviderStatusResult>;
  refund(request: RefundRequest): Promise<ProviderRefundResult>;
  disburse(request: DisbursementRequest): Promise<ProviderPayoutResult>;
  normalize(payload: unknown): NormalizedTransaction;
  verifyWebhook(req: Request): boolean;
}
```

LOOP Adapter — implements authentication, the NEO Merchant Request-to-Pay flow, status polling/webhook handling, and normalization, plus disburse() against LOOP's payout/B2C endpoint. Exact endpoint paths, auth headers, and payloads are pulled from the LOOP developer sandbox at build time rather than assumed.

Seeded/Future-Rail Adapter — implements the identical interface against static/seeded fixtures. This is the piece that proves the "unified, scalable, currency-ready" claim without needing a second live integration in this phase.

Identity verification is deliberately not forced into this interface — it isn't a payment rail, so it gets its own lightweight service (Section 16) rather than being awkwardly bolted onto the adapter contract.

11. Data Model

```
profiles
  id, account_type (individual|business), display_name, owner_name,
  clerk_user_id, phone, email, currency (default 'KES'), country_code,
  status, verification_status, id_number, id_document_url,
  id_submitted_at, id_reviewed_at, id_reviewer_note, id_ai_check_result,
  created_at

aliases
  id, profile_id, alias, identifier_type, is_verified, status

payment_intents
  id, recipient_profile_id, order_reference, amount, currency,
  payer_phone, payer_email, provider, rail, status, provider_reference,
  idempotency_key, expires_at, initiated_at, completed_at

transactions
  id, recipient_profile_id, payment_intent_id, provider, rail,
  internal_reference, external_reference, amount, currency,
  provider_fee, net_amount, payer_identifier, payment_status,
  settlement_status, refund_status, transaction_time, settled_at,
  ai_category, raw_payload

settlements
  id, profile_id, provider, settlement_reference, currency,
  gross_amount, fees, net_amount, status, expected_at, settled_at

reconciliation_matches
  id, profile_id, transaction_id, expected_reference, expected_amount,
  matched_amount, match_type, confidence_score, ai_explanation,
  status, notes

payouts
  id, profile_id, provider, requested_amount, requested_currency,
  destination_type, destination_reference, fee, net_amount, status,
  provider_reference, requested_at, processed_at, raw_payload

money_direction_rules
  id, profile_id, destination_type, destination_reference,
  allocation_type (full|percentage|fixed_amount), allocation_value,
  priority_order, is_active, updated_at

payment_rails
  id, name, adapter_key, is_enabled, supported_currencies,
  supported_countries, min_amount, max_amount, capabilities_json

admin_users
  id, clerk_user_id, role (super_admin|support|compliance_reviewer),
  permissions_json, created_at

audit_logs
```

```
  id, actor_type (admin|system|user), actor_id, action, target_type,
  target_id, before_state, after_state, created_at

ai_interactions
  id, profile_id, interaction_type (query|support|reconciliation|
  document_check|fraud_flag), input_summary, output_summary,
  confidence_score, reviewed_by_human, created_at
```

payouts stays a separate table from **settlements**: settlements records money the provider confirms as settled into UniPay's/the user's provider balance; payouts records the actual movement of that balance to wherever the user's money-direction rule sends it. **money_direction_rules** is new in this version — it's what lets a user say "send 70% straight to my LOOP number and keep 30% in my UniPay balance," and change that split later without re-doing onboarding (Section 17). **admin_users** and **audit_logs** back the Admin module (Section 16); **ai_interactions** gives a full, reviewable trail of every AI-assisted decision the platform makes (Section 15).

12. Payment Lifecycle

Flow: Alias → Recipient resolution → Amount entry → LOOP request-to-pay → Payer approves on their phone → Webhook/poll → Normalized transaction → AI-assisted reconciliation → Money-direction routing → Dashboard.

Payment vs. settlement vs. destination: the system tracks these as three related but independent states at all times. A transaction can show *payment_status: successful* while *settlement_status: pending*, and once settled, its net amount is routed according to the user's active money_direction_rules — which may still leave part of it "in transit" to a chosen destination. These are shown as distinct badges everywhere in the UI, never collapsed into one status field.

Disbursement flow: once transactions are settled, their net amounts contribute to a user's available-to-withdraw balance. Money-direction rules can trigger this automatically, or the user can tap "Withdraw" manually and specify an amount up to that balance. Either path creates a payouts record with status requested → processing → completed/failed, calling the LOOP adapter's disburse() method.

13. Fees & Transparency

Shown to the payer before they confirm, and to the recipient on every transaction row:

Net Amount = Amount − Provider Fee − Platform Fee − Tax

No unexplained flat discounts — UniPay always shows the real cost breakdown and lets the recipient decide whether to absorb or pass on fees. Because this phase is KES-only, no FX component applies; the fee model retains an fx_fee field at the data layer so Phase 2 currency support requires no schema change (Section 25).

14. Reconciliation Engine

Matching rules, in priority order:

- Exact invoice/order reference match — highest confidence
- Exact amount within a configured time window
- Payer phone or email + amount
- AI-assisted fuzzy match — for near-miss references (typos, partial invoice numbers, informal descriptions like "rent aug"), the AI layer proposes a match with a confidence score and a plain-language reason
- Manual review when no rule clears a confidence threshold

Exception categories: missing provider transaction, missing order, amount mismatch, duplicate payment, fee mismatch, settlement delay, unknown provider reference.

Dashboard surfaces: gross collections, net collections, total fees, successful payments, pending settlements, failed payments, reconciliation rate, open exceptions, and a downloadable report.

15. AI Integration — The Intelligence Layer

This is treated as a first-class architectural layer, not a feature. AI is integrated through one internal service — the **AI Service** — that every other module calls through a narrow, well-defined interface. This keeps AI genuinely useful (it has access to real transaction context) while keeping it safely bounded (it never directly moves money, changes a status, or approves an identity on its own — every AI output that affects money or trust is either a suggestion a human/rule confirms, or an assist a human reviews).

```
interface AIService {
  explainMatch(match: ReconciliationMatch): string;                 // P0
  answerDashboardQuery(profileId: string, query: string): QueryAnswer; // P0
  flagAnomalousActivity(profileId: string,
     recentTx: NormalizedTransaction[]): AnomalyFlag[];             // P1
  suggestRailRouting(context: RoutingContext): RailRecommendation;  // P1
  precheckIdDocument(imageUrl: string, claimedFields: IdFields):
     DocumentCheckResult;                                           // P1
  generateSummary(profileId: string, period: DateRange): string;    // P1
  draftSupportReply(conversation: SupportMessage[]): string;        // roadmap
  predictSettlementDelay(tx: NormalizedTransaction): DelayForecast; // roadmap
}
```

Every method maps to one of the LLM integrations below, prioritized by build effort against demo impact — deliberately mirroring how the payment-rail adapters are prioritized in Section 8 (Must-Build vs Should-Build vs Roadmap), so AI features are held to the same honesty standard as everything else in this document: nothing is presented as live if it isn't.

Priority 0 — Must-Build (one LLM call each, highest demo impact)

These two are the AI features the platform actually builds and demos live. Both are cheap (a single LLM API call per interaction), both are visible to the user in under 30 seconds, and both directly reinforce the platform's core differentiator — reconciliation and settlement truth as the product, not collection alone.

1. LLM-explained reconciliation matches

The reconciliation engine (Section 14) already produces a numeric confidence score per match. Rather than showing Amina a bare number, an LLM call converts the match's underlying signals into one plain-language sentence — turning a black-box score into something a non-technical user actually trusts, instead of re-checking by hand.

```
// AIService.explainMatch — called once per newly created match
POST https://api.anthropic.com/v1/messages
{
  "model": "claude-sonnet-4-6",
  "max_tokens": 100,
  "system": "You write a single, plain-language sentence explaining",
  ..."why a payment was matched to an expected order. Be concrete: name",
  ..."which signals matched (amount, reference, phone, time window) and",
  ..."state the confidence level in everyday words. One sentence only.",
  "messages": [{
    "role": "user",
    "content": JSON.stringify({
      transaction: { amount: 3000, currency: "KES",
        payer_identifier: "2547XXXXXXXX", reference: "grocries amina",
        received_at: "2026-08-13T09:14:00Z" },
      expected_order: { reference: "AMINA-GROCERIES-0417",
        amount: 3000, expected_around: "2026-08-13T09:10:00Z" },
      match_signals: { amount_exact: true, reference_fuzzy_score: 0.62,
        time_window_minutes: 4 },
```

```
      confidence_score: 0.91
    })
  }]
}

// Example output rendered next to the match in the dashboard:
// "Matched on exact amount (KES 3,000) and a 4-minute time window —
//  the reference was a near-match, so this is a high-confidence match."
```

This is a direct extension of existing infrastructure — the confidence score and match signals already exist in reconciliation_matches (Section 11); the AI Service only adds a stored ai_explanation string, logged in ai_interactions for auditability.

2. Natural-language dashboard queries

A search box on the dashboard lets any user type a question in plain English or Swahili — "how much did I make last week", "which payments are still unsettled" — which the AI Service translates into a structured, parameterized query against the (small, well-defined) transaction schema, rather than a free-form database call. Because the schema is fixed and small, this is a low-integration-risk, high-payoff feature.

```
// AIService.answerDashboardQuery
POST https://api.anthropic.com/v1/messages
{
  "model": "claude-sonnet-4-6",
  "max_tokens": 300,
  "system": "Translate the user's question into a JSON filter against",
  ..."the `transactions` table (fields: amount, currency, payment_status,",
  ..."settlement_status, transaction_time, recipient_profile_id). Return",
  ..."ONLY JSON: {filters, aggregation, explanation}. Never invent fields.",
  "messages": [{
    "role": "user",
    "content": "How much did I make last week compared to the week before?"
  }]
}

// Response is parsed, run against the transactions table server-side
// (the model never touches the database directly), and the result plus
// the model's one-line explanation are shown together — so the answer
// is auditable, not a black box.
```

Priority 1 — Should-Build / "AI-ready" (same architecture, built if time allows)

These extend the same AIService interface and the same exception-queue/audit-log infrastructure already built for the two P0 features above — they are a config/prompt addition, not a new subsystem, which is why they're realistic to demo even if only partially wired up:

- **Anomaly / fraud flagging.** A lightweight check (a simple statistical threshold, or a prompt fed transaction metadata) flags patterns like "this alias never receives more than KES 5,000 — this transaction is 10x that." Routes into the same exception queue reconciliation already uses, extending the fraud/velocity item already on the roadmap.
- **AI-assisted smart rail routing.** Rather than a static rail ranking, a weighted-scoring function — framed and logged as an AI-assisted recommender — considers historical success rate, fee, and settlement speed per rail to recommend the best option. Logging outcomes over time lets this genuinely improve, and it's a natural extension of the money-direction work already in Section 17.
- **ID verification pre-check.** Vision-model extraction of the ID number and name from an uploaded document, cross-checked against what the user typed. This is explicitly triage, not registry verification — an honest, scoped use of AI that flags obvious mismatches or blurry uploads before the (simulated) review queue, consistent with the honesty boundary in Section 19.

- **Receipt / summary generation.** An LLM call over a user's transaction data generates a plain-language weekly summary ("You received 12 payments this week, KES 45,000 net, 2 exceptions resolved automatically") or a customer-facing receipt — cheap, demoable, and consistent with the platform's "one-glance" UX principle (Section 26).

Roadmap only — named, not built

Two AI ideas are worth naming for forward-thinking credit but are deliberately not built in this phase, since neither reinforces the core reconciliation/trust differentiator as directly as the P0 features, and one requires historical data this phase doesn't yet have:

- **Predictive settlement-delay warnings** ("this transaction is likely to settle 2 days late based on historical patterns") — needs a real history of settlement timing this phase hasn't accumulated yet; listed in Section 25.
- **Chat-based merchant support bot** trained on UniPay's own docs/FAQ — a reasonable nice-to-have, but lower priority than the reconciliation- and query-focused features above, so it stays a roadmap item rather than a build target.

Why this prioritization is the right shape for AI here

- It is scoped to assist, not decide, everywhere money or identity is on the line — the product stays trustworthy even if a model is occasionally wrong.
- The two P0 features are each a single LLM API call, so they are cheap to build and demo reliably, rather than spreading limited build time across many shallow AI touchpoints.
- P1 items reuse the same interface, exception queue, and audit trail as the P0 features, so naming them as "AI-ready, same adapter-style architecture" is an architectural fact, not a marketing line.
- It is fully logged (ai_interactions table, Section 11), so every AI-influenced outcome is auditable after the fact — a requirement for anything touching payments.
- It is provider-agnostic at the integration boundary — the AIService interface can be backed by any capable LLM API (e.g. Claude) without touching any calling code, the same pattern the payment adapters use for rails.

16. Admin Module

UniPay's operations team needs the same visibility and control over the platform that users have over their own accounts — without giving them raw database access. The Admin module is a separate, role-gated section of the same application (not a separate app), authenticated via Clerk's organization/roles feature and backed by the admin_users and audit_logs tables (Section 11).

Core Admin Capabilities

- **User & identity management** — search and view any profile (individual or business), review the identity-verification queue (including the AI pre-check result from Section 15), approve/reject/suspend accounts, and see a profile's full transaction history.
- **Transaction & exception oversight** — a platform-wide view of transactions, settlements, and open reconciliation exceptions, filterable by status, rail, date range, or AI confidence score, with the ability to manually resolve or escalate an exception.
- **Rail & configuration control** — enable/disable rails and currencies, adjust fee configuration, and manage the payment_rails table entirely through UI, with every change written to audit_logs (this is also the operational surface for Section 9b's "rails as configuration" claim).
- **Payout & dispute handling** — visibility into every payout request across the platform, ability to intervene on a stuck or failed payout, and a lightweight dispute/complaint queue.
- **Reporting & platform health** — aggregate metrics (volume, reconciliation rate, exception rate, AI-suggestion acceptance rate) and basic rail health indicators (last successful call, error rate per adapter).

- **Role-based access** — distinct admin roles (super_admin, support, compliance_reviewer) each see and can act on only what their role permits, enforced server-side, not just hidden in the UI.

Every admin action that changes user-visible state — approving an identity, editing a fee, resolving an exception — is written to audit_logs with before/after state, actor, and timestamp, so the platform's own operators are as accountable as its users.

17. Money Direction — User-Controlled Payout Routing

Where settled money goes has, in earlier versions of this platform, been an implicit default (everything sits in a single balance until manually withdrawn). This version makes it an explicit, user-owned setting: every user chooses where their money goes, and can change that choice at any time from Settings — it is never a one-time onboarding decision.

How it works

- **Destination options (this phase):** keep as UniPay available balance (default), auto-forward to the LOOP-linked mobile number on file, or split between the two by percentage or fixed amount.
- **Multiple active rules with priority.** A user can define more than one money_direction_rules row (e.g. "send the first KES 5,000 of any settlement to my LOOP number, keep the rest as balance"), with a priority_order UniPay applies in sequence as funds settle.
- **Changeable at any time.** Editing money-direction rules in Settings takes effect on the next settlement — it never touches funds that have already been routed, keeping the payout history honest and auditable.
- **Same mechanism for individuals and businesses.** A business owner splitting revenue between operating balance and immediate cash-out, and an individual who wants family-sent money to land directly on their phone, use the identical settings screen and the identical money_direction_rules schema.
- **Built to extend.** Because destination_type is a config value rather than hardcoded logic, adding a bank account, a savings product, or (post multi-currency) a foreign-currency wallet as a future destination is additive, following the same adapter/config pattern used for rails (Section 9b).

18. API Design (Core Endpoints)

```
POST   /api/v1/profiles                          # account_type: individual|business
POST   /api/v1/profiles/:id/aliases
GET    /api/v1/aliases/:alias
POST   /api/v1/checkout/payment-options
POST   /api/v1/payment-intents
GET    /api/v1/payment-intents/:id
POST   /api/v1/payment-intents/:id/retry
POST   /api/v1/webhooks/loop
GET    /api/v1/transactions
POST   /api/v1/reconciliation/run
GET    /api/v1/reconciliation/exceptions
GET    /api/v1/exports/transactions.csv
POST   /api/v1/profiles/:id/identity          # submit ID number + document
GET    /api/v1/profiles/:id/identity          # check verification_status
POST   /api/v1/profiles/:id/identity/review   # (admin) approve or reject
GET    /api/v1/profiles/:id/balance           # available-to-withdraw balance
GET    /api/v1/profiles/:id/money-direction   # current routing rules
PUT    /api/v1/profiles/:id/money-direction   # update routing rules
POST   /api/v1/payouts                        # request/trigger a disbursement
GET    /api/v1/payouts/:id
GET    /api/v1/payouts
POST   /api/v1/ai/query                       # natural-language dashboard query
POST   /api/v1/ai/support                      # AI support chat turn
GET    /api/v1/admin/users
GET    /api/v1/admin/exceptions
```

```
PUT    /api/v1/admin/payment-rails/:id         # enable/disable/configure a rail
GET    /api/v1/admin/audit-logs
```

Example — payment options request/response:

```
// Request
{ "alias": "@amina", "amount": 3000, "currency": "KES" }

// Response
{
  "provider": "loop",
  "rail": "request_to_pay",
  "amount": 3000,
  "currency": "KES",
  "estimated_fee": 15,
  "estimated_recipient_amount": 2985,
  "settlement_estimate": "instant"
}
```

19. Authentication & Security

- Clerk handles sign-up, sign-in, session management, and password/email flows entirely for every user, individual or business; the backend only trusts Clerk-issued JWTs on protected routes.
- Payer-facing checkout stays unauthenticated by design — requiring an account before paying is a conversion killer and out of scope for what UniPay is solving.
- LOOP secrets and AI-provider API keys stay server-side only, never shipped to the React client.
- All traffic over HTTPS; webhook signatures verified; webhook event IDs deduplicated; idempotency keys on payment intent creation.
- No card numbers or sensitive payment credentials stored anywhere in UniPay's own database.
- Identifiers (phone/email) masked in logs; every AI interaction and every admin action is written to an append-only audit log.
- AI Service calls are scoped per-request to a single profile's data — no cross-profile context is ever passed to a model call, and AI outputs affecting money or identity are always confirmed by a rule or a human before taking effect.
- Positioned explicitly as an orchestration layer over an authorized provider, not a deposit-taking or lending service — important both for real-world viability and for evaluators assessing compliance awareness.

Identity Verification (Lightweight, Realistic Scope)

Full KYC — live registry checks, biometric matching, sanctions screening — is not in scope for this phase, and claiming otherwise would be dishonest and unsafe. What is built is a submit-and-track verification flow, AI-assisted at the pre-check step only:

- During onboarding, the user uploads a government-issued ID and enters their ID number; the AI Service runs a legibility and field-consistency pre-check.
- The record is created with verification_status: submitted; the alias/QR is created but marked unverified until review completes.
- For the demo, review is auto-approved after a short delay or manually approved via an admin toggle — both are labeled honestly as simulated review, not a live registry check.
- Once approved, verification_status: verified unlocks the visible checkmark on the user's public checkout page.
- ID numbers and document URLs are treated as sensitive data: never logged, never exposed beyond the user's own authenticated dashboard or the Admin module, and excluded from CSV exports.

Production deployment would require integrating a licensed ID-verification provider (e.g., Smile Identity, which supports Kenyan national ID verification) rather than UniPay's own review queue.

20. Unified Web & Mobile Experience

UniPay is one application, not two. There is no separate native mobile codebase in this phase — the same React + Tailwind frontend renders as a full desktop dashboard on a laptop and as a mobile-optimized layout on a phone, using responsive breakpoints rather than device-specific builds.

- Checkout and onboarding are designed mobile-first, since most payers and many users will interact with UniPay primarily from a phone; the desktop dashboard is a wider-viewport expansion of the same components, not a separate design.
- QR-code and alias-link entry points work identically on both — scanning a QR on a phone or clicking a link on a desktop lands on the same responsive checkout page.
- This avoids the classic split-team trap of maintaining a web app and a native app in parallel with two codebases and two release cycles, which is not a good use of a small team's time at this stage.
- A installable Progressive Web App (PWA) wrapper — giving an app-like icon and offline-tolerant shell without a native build — is a natural, low-effort next step and is noted on the roadmap (Section 25) rather than built now.

21. Competitive Landscape & Why UniPay Is a Better Solution

UniPay does not compete with LOOP, M-Pesa, or other rails — it sits above them. But it does compete with how Kenyans currently experience payments and money management. This section compares UniPay against the real systems people use today.

System	What it does well	Where it falls short
M-Pesa Paybill / Till	Ubiquitous, trusted, instant local transfers	No unified view across till + paybill + personal number; no settlement-vs-paid distinction; no reconciliation; business and personal use are awkwardly mixed on one line
Bank mobile apps (e.g. equity, KCB apps)	Strong for holding and moving larger balances, has statements	Not built for a market-stall checkout experience; no alias/QR-first flow; reconciliation against orders is manual
Generic payment links (e.g. simple till/paybill sharing via WhatsApp)	Zero setup, familiar	No identity verification at all — trivially spoofable; no fee transparency before payment; no ledger, just a stream of SMS alerts
International gateways (Stripe, Flutterwave, PayPal)	Strong developer tooling, multi-currency, good documentation	Built for developers integrating a checkout, not for a non-technical vendor or an ordinary person; no concept of a personal, general-population identity; overkill for a KES-only local transaction
Informal tracking (notebooks, WhatsApp, Excel)	Free, flexible, already how many small sellers operate	Entirely manual, error-prone, not exportable, no verification, no automation, doesn't scale past a handful of transactions a day

UniPay's improvement over each

- **Versus mobile-money numbers/tills:** one verified identity instead of several disconnected identifiers, plus an automatic ledger and reconciliation layer mobile money never provided on its own.
- **Versus bank apps:** purpose-built for the alias/QR checkout moment and for small, frequent transactions, with reconciliation and settlement-status logic banking apps don't attempt.

- **Versus informal payment links:** identity verification and fee transparency close the two biggest trust gaps in how Kenyans currently share till/paybill numbers informally.
- **Versus international gateways:** built for the actual target user — a non-technical vendor or an ordinary person — not a developer integrating an SDK, while retaining the same adapter-based architecture those platforms use internally, so it can grow toward their capability without starting over.
- **Versus notebooks and WhatsApp tracking:** everything those methods do manually — remembering who paid, matching payments to orders, tallying totals — UniPay's reconciliation engine and AI layer do automatically, while remaining as approachable as a QR code.

The core differentiator that no listed alternative offers together: one identity usable by anyone (not just registered merchants), explicit payment-vs-settlement truth, AI-assisted reconciliation, user-controlled money direction, and platform-level administrative oversight — in a single, unified product.

22. Gaps in the Original Spec — Identified & Addressed

Gap	Risk if unaddressed	Fix in this version
No authentication strategy specified	Team burns time building custom auth instead of payment logic	Clerk adopted for all users; payer checkout stays guest
Platform was merchant-only	Excludes the majority of the population who aren't registered sellers, limiting real-world reach	Single account model with an account_type flag serves individuals and businesses identically (Section 7, 9b)
No distinction between "live" and "seeded" rails	Evaluators may probe a rail assumed to be real and expose a mock	Seeded/future rail explicitly labeled and framed as a scalability proof, not a hidden fake
No mobile-first consideration, or risk of building two separate apps	Most users pay from a phone; a desktop-only or dual-codebase approach undermines usability and burns build time	Single responsive codebase, mobile-first Tailwind breakpoints (Section 20)
No verification signal for aliases	Anyone could claim @amina — a real fraud vector	ID upload + ID number captured at onboarding, verification_status gates the visible checkmark, AI pre-check assists (but does not replace) human review
No disbursement/payout capability	Collection-only tooling doesn't solve the real problem — money needs to become spendable, and go where the user wants	payouts table, dashboard withdraw action, and user-defined money_direction_rules (Section 17)
No administrative control surface	Operators would need direct database access to manage users, exceptions, or configuration — unsafe and unscalable	Dedicated, role-gated Admin module with full audit logging (Section 16)
No meaningful AI integration	Reconciliation, support, and identity review remain fully manual bottlenecks as volume grows	A bounded AIService layer integrated across reconciliation, dashboard queries, document pre-check, fraud flags, and support (Section 15)
Multi-currency assumed from day one	Spread effort thin across currency/FX logic before the core single-currency product was solid	Phase 1 intentionally scoped to KES only; schema and adapter pattern already accommodate future currencies without rearchitecture (Section 25)

23. Viability & Real-World Usability Assessment

Is the problem real? Yes — fragmented identifiers and manual reconciliation are well-documented pain points across Kenyan mobile-money and informal commerce, and they affect ordinary individuals sending and tracking money just as much as registered merchants.

Is the solution technically viable beyond this phase? Yes, with caveats: the adapter pattern is a standard, production-proven approach to multi-provider orchestration. The main production gaps are KYC/compliance depth, licensing to legally sit in a payment flow at scale, and building out real additional rails and currencies — all correctly scoped as roadmap rather than ignored.

Is it usable by the actual target population? Amina, Ken, and similar users are not technical — the design choices that matter most are: (1) QR/alias-first checkout over app installs, (2) plain-language fee display instead of financial jargon, (3) a dashboard and AI assistant that answer "did I get paid and can I spend it" in one glance or one question, (4) no forced distinction between "consumer" and "merchant" apps. These are treated as core requirements, not polish.

What would break it in the real world first? Verification and trust — an unverified alias system is trivially spoofable. This remains the top production risk and should be the first thing hardened post-launch, ahead of adding more rails or currencies.

24. Build Plan

Phase	Focus	Outcome
1	Repo, React + Tailwind responsive scaffold, Postgres/Supabase schema, Clerk integration (single account model)	Any user can sign up and log in, individual or business
2	Onboarding: ID upload + AI pre-check, verification_status flow, simulated review toggle	Identity verification pillar demoable
3	Profile, alias creation, QR generation, verified checkmark	Any user has a trusted, shareable identity
4	LOOP sandbox auth + request-to-pay + status polling + normalization	Real local payment works end-to-end, KES only
5	Normalized transaction ledger + payment/settlement status split	One truthful record per payment
6	Checkout page (alias resolve → amount → LOOP pay), mobile-first responsive layout	Payer-facing collection flow complete on web and mobile
7	Money-direction settings + disbursement: payouts table, dashboard withdraw action, LOOP payout adapter	User-controlled payout routing pillar demoable
8	AI Service: reconciliation-assist, dashboard NL queries, categorization	Intelligence layer demoable
9	Rule-based + AI-assisted reconciliation engine + seeded future-rail proof	Auto-matching demoable, scalability proven
10	Unified dashboard (React): totals, balance, payouts, exceptions + CSV export	Judged deliverable: unified view for any user type
11	Admin module: user/identity queue, exception oversight, rail config, audit logs	Administrative control surface demoable
12	Seed demo data, rehearse demo narrative, fix rough edges	Believable, repeatable demo

13	Record backup demo video, prep submission package	Safety net if live demo hardware/network fails

25. Post-Phase-1 Roadmap

- Multi-currency support and cross-border rails (PayPal, international cards), enabled through the existing payment_rails configuration rather than a rewrite
- Real alias verification against a licensed registry provider (e.g. Smile Identity)
- USSD/SMS fallback checkout for low-connectivity users
- Redis-backed queue for webhook processing and retries at scale
- Additional real rails (bank A2A, card acquiring) behind the existing adapter interface
- Progressive Web App packaging for an installable, app-like mobile experience without a native build
- Predictive settlement-delay warnings, once enough real settlement history has accumulated to make the forecast meaningful (Section 15)
- Chat-based merchant support bot trained on UniPay's own documentation and FAQ (Section 15)
- Deeper AI capabilities: proactive cash-flow insights, personalized savings nudges tied to money-direction rules, and multilingual support conversations
- Expanded Admin module: role customization, bulk operations, configurable alerting

26. Additional Features Worth Considering

Beyond what's scoped for this phase, a few features would materially strengthen the platform's unified story:

- **Smart rail/destination routing.** Once more than one rail or destination exists, the AI Service could recommend the best option automatically based on cost, speed, and past behavior — turning money direction from "here are your choices" into "here's your best choice."
- **Sub-accounts / staff access.** Read-only or limited-scope logins (via Clerk's organization/roles feature) would make UniPay usable by a small team, not just a solo owner or individual.
- **Customer-facing receipts.** An SMS or email receipt on successful payment, referencing the same normalized transaction record.
- **Rail health monitoring for users.** A simple status indicator so a user knows why a rail might be temporarily unavailable rather than seeing an unexplained failure — an extension of the Admin module's internal health view.
- **Developer API / webhooks.** Letting a business's own website or POS system subscribe to "payment received" events — a natural extension of the webhook handling already built for LOOP.
- **Basic fraud/velocity checks beyond AI flags.** Simple deterministic rules layered onto the existing exception-handling pattern, reusing infrastructure rather than adding a new subsystem.

27. Demo Narrative

Amina signs up on UniPay as a Business, uploads her ID, and submits her details — her alias goes live as unverified. The AI pre-check flags her ID as legible and consistent, and minutes later her verification clears; a checkmark appears next to @aminagroceries. A customer scans her QR code and pays via LOOP in KES — confirmation arrives in seconds.

Separately, Ken — an ordinary UniPay user with an Individual profile — sends his friend rent money using the same alias system, no business account needed.

Amina opens her UniPay dashboard and asks the AI assistant, in plain language, how her week compares to last week. She sees a payment with a slightly mismatched reference that the AI has already matched with a confidence score and an explanation. Her money-direction setting sends 70% of settled funds straight to her LOOP number and keeps 30% as UniPay balance — she taps to adjust that split for the week of a big restock, then exports everything as one CSV.

Meanwhile, a UniPay Admin reviews the day's identity-verification queue, checks the reconciliation exception rate on the platform dashboard, and confirms the LOOP rail's health — all without touching a database.

Verified identity → any-user payment (individual or business) → transparent fees → AI-assisted reconciliation → user-controlled money direction → unified dashboard → admin oversight → exportable report — in one platform, for everyone.

28. Evaluation Criteria Alignment

Criterion	Evidence
Technical Implementation	Live LOOP sandbox integration (collection and disbursement), Clerk-authenticated unified account flow, ID-upload identity submission with AI pre-check, normalized ledger, working reconciliation and export, functioning Admin module
Innovation & Scalability	Adapter-based architecture proven with a seeded second rail/currency; account-type flag proven to unify individuals and businesses without a fork; bounded AIService layer integrated across four+ real use cases
User Experience	Mobile-first responsive checkout (one codebase), verified-identity trust signal, plain-language fees, one-glance dashboard with AI natural-language queries, user-controlled money direction
Market Impact	Serves the entire population, not a merchant subset — vendors, freelancers, individuals sending money, and online sellers — solving "how do I get paid," "how do I trust who I'm paying," and "where does my money go"
Presentation	Coherent multi-persona narrative (Amina, Ken, Admin) carried through problem → build → demo → roadmap
Administration & Governance	Dedicated, role-gated Admin module with full audit logging — platform is operable and accountable, not just demoable

29. Value Score Justification

- **Value:** Cuts manual reconciliation effort and eliminates "did I actually get paid" uncertainty for anyone handling money, not just registered merchants
- **Integration:** Real LOOP sandbox integration, a bounded AI service integrated across reconciliation/support/queries/document-checks, and an Admin control plane, all behind clean interfaces
- **Build Effort:** A real local rail, a proven-scalable seeded rail/currency, a functioning AI layer, and an Admin module — a broader platform than a checkout page, delivered on a tight build
- **Edge:** Universal (not merchant-only) identity, AI-assisted reconciliation, user-controlled money direction, and explicit payment/settlement/destination separation
- **Partnership:** LOOP positioned as an infrastructure partner, not a competitor, with UniPay adding coordination, intelligence, and administrative value on top

---

This document is a phased architecture and implementation specification. Production compliance, licensing, KYC, and settlement contracts require direct review with LOOP and relevant regulators before any real-world deployment.